Факультет Компьютерных Наук

Москва
2026

Optimized Visual Recognition Through a Deep Convolutional Neural Network With Hierarchical Modular Organization

Журнал: IEEE Access
Авторы: ADE CLINTON SITEPU, AND CHUAN-MING LIU
Презентующие: Горельский Роман, Нечаева Ксения, Поздеева Мария

Содержание

Subject Area

HMDCNN-Tree

Efficient visual recognition with deep convolutional neural networks

Example tasks: image classification, object detection, image segmentation, image enhancement, hyperspectral image classification, ...

Examples in real life: autonomous driving, medical imaging, face recognition, on-device / edge vision, ...

Rapid increase in computational cost as network depth and the number of classes grow.

Relevance

- **Reduction** of redundant computation in over-parameterized **monolithic** DCNNs
- High potential for a **hierarchical modular** organization to be **improved** with **selective module activation** to fight the **challenge** of computational redundancy

Literature review

Modularity and hierarchical organization in neural networks

“A review of modularization techniques in artificial neural networks”

Amer, Mohammed and Maul, Tomas (2019)

A taxonomy of modularization techniques; the HMDCNN-Tree is built as learned domain, multi-path topology and manual formation

“Design of a hierarchy modular neural network and its application in multimodal emotion recognition”

Li, Wenjing and Chu, Mingyang and Qiao, Junfei (2019)

Multiple small subnetworks are arranged into a hierarchy, each specializing in an intermediate classification step

“Deep Residual Learning for Image Recognition”

He, Kaiming and Zhang, Xiangyu and Ren, Shaoqing and Sun, Jian (2016)

Introduces residual connections that enable training of much deeper networks; ResNet18 is used here as a strong baseline

“Very Deep Convolutional Networks for Large-Scale Image Recognition”

Simonyan, Karen and Zisserman, Andrew (2015)

Introduces the VGG family using stacked 3×3 convolutions; VGG16 is used here as a monolithic baseline

Постановка проблемы

Метрика схожести картинок

$$\sigma(\vec{z})_i = \frac{\exp(z_i)}{\sum_{j=1}^{n} \exp(z_j)} \longrightarrow L(A)_B = \frac{\sum_{i=0}^{|A|} \sigma(\vec{A})_B}{|A|}$$

Второе уравнение описывает, как вычисляется MSL: вектор A представляет входной вектор необработанных выходных данных из DCNN, где A= (A0,A1,...,A|A|), и он был ошибочно отнесен к классу B. |A| - это общее количество входных изображений, помеченных как class A в узле. σ(A)B представляет все значения softmax экземпляров из класса A, которые ошибочно классифицированы как класс B.

Постановка проблемы

Применение нечеткости

Сигмоидальная функция принадлежности (sgmf) оценивает степень принадлежности для каждого класса, а выходные данные функции принадлежности указывают на вероятность присутствия определенного класса в суперкластере. С a в качестве параметра наклона и b в качестве центрального параметра, который определяет точку максимального членства в контексте нечеткой логики, sgmf может быть определен как:

$$\mu(o) = \frac{1}{1 + \exp\left(-a(o - b)\right)}$$

$$p(A \sim B) = \mu(L(A)_B)$$

$$= \frac{1}{1 + \exp\left(-10M \times (L(A)_B - \frac{1}{M})\right)}$$

Постановка проблемы

Описание алгоритма (выбор размера свёрточных слоёв, детекция супер-кластеров)

Algorithm 1 Building and Configuring HMDCNN-Tree

inputs: Image or output feature map from the parent module
output: Trained HMDCNN-Tree structure T
initialization:
$T \leftarrow$ Untrained root DCNN ▷`Set containing the tree structure`
algorithm:
$\mathbb{C} \leftarrow$ set of all classes at the root
while ($\exists$ a node in T containing an untrained DCNN)

1) $lconv \leftarrow 1$ ▷`number of conv layers`
2) **do**
   - $lconv' \leftarrow lconv - 1$
   - Initialize a DCNN D_{lconv}
   - Train D_{lconv} to classify all classes of $\mathbb{C}$
   - **if** $lconv > 1$ **then** calculate:
     $\Delta EG_{lconv,lconv'} = \frac{VA_{lconv} - VA_{lconv'}}{MS_{lconv} - MS_{lconv'}}$
   - $lconv \leftarrow lconv + 1$

   while ($lconv' == 2 || \Delta EG_{lconv,lconv'} > \tau$)
3) $DCNN_{config} \leftarrow D_{lconv'}$ ▷ `Select previous DCNN configuration for module`
4) $SOFTMAX \leftarrow$ Softmax output of $DCNN_{config} \forall c \in \mathbb{C}$ ▷ `Softmax output matrix`
5) $CLUSTER \leftarrow \phi$ ▷`Track new children's clusters`
6) **for each** $c \in \mathbb{C}$ ▷`Use the MSL`
   - Find set $\mathbb{H}$, such that $\mathbb{H} \subset \mathbb{C}$: $\text{prob}(c \sim h) = \mu(SOFTMAX_{[c,h]}), \forall_{h,c} \in \mathbb{H}$
   - Add untrained DCNN corresponding to $\mathbb{H}$ into T
   - $CLUSTER \leftarrow CLUSTER \cup \mathbb{H}$ ▷`Add similar classes to the children's set`
7) Train $DCNN_{config}$ to classify between elements of $CLUSTER$
8) $T \leftarrow T \cup$ trained $DCNN_{config}$
9) $\mathbb{C} \leftarrow$ set of all classes at next node

return T

Методика и результаты

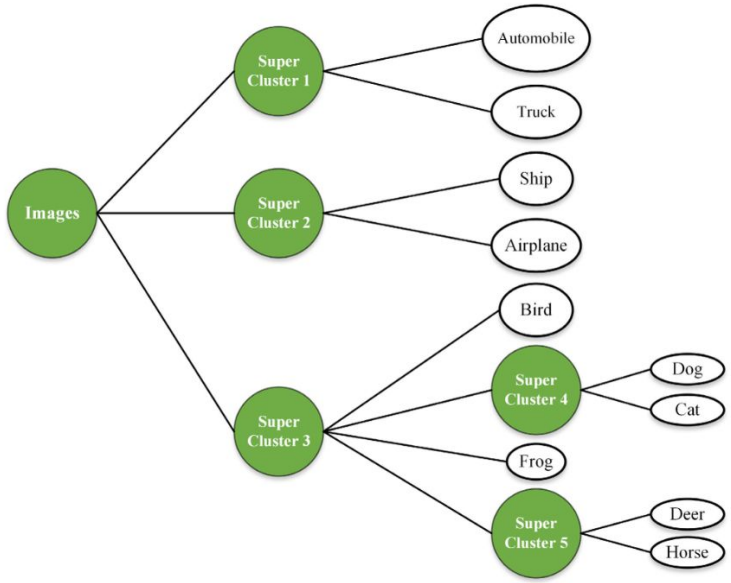
Использованные данные

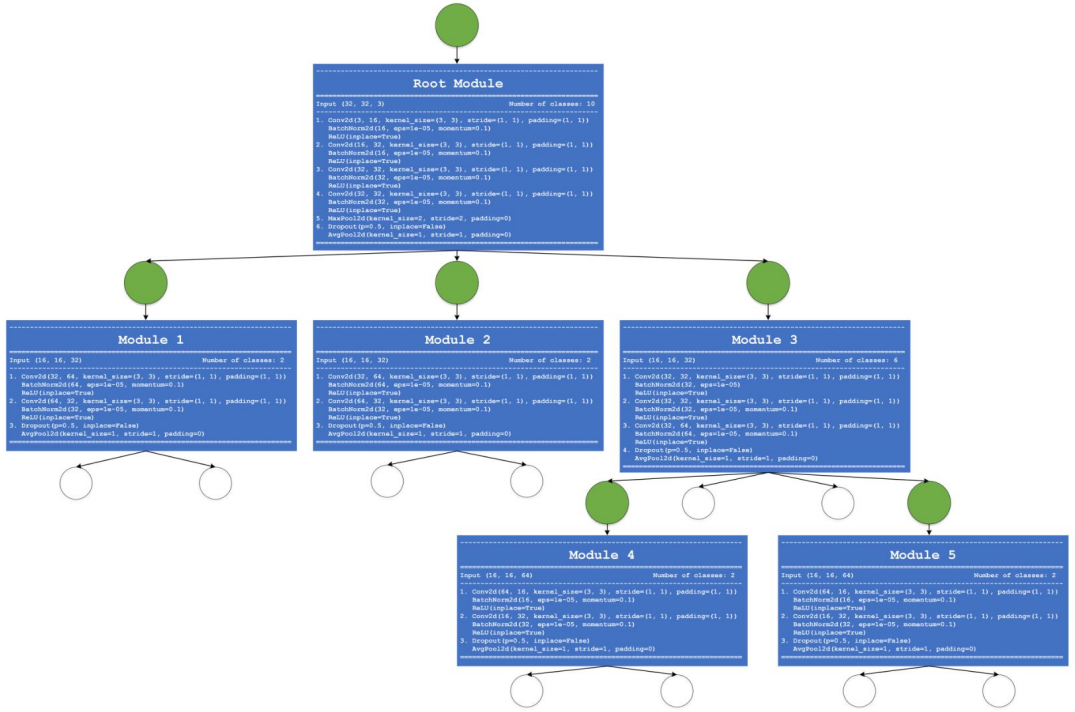
Dataset	Image Size ($W \times H \times C$)	No. of Training Images	No. of Test Images	No. of Classes
CIFAR 10	$32 \times 32 \times 3$	50,000	10,000	10
EMNIST	$28 \times 28 \times 1$	112,800	18,800	47
SVHN	$32 \times 32 \times 3$	604,388	26,032	10

FIGURE 5. Selected images from the different datasets used in the experiments.

Методика и результаты

Методика





1. После обучения HMDCNN-Tree точность классификации оценивается с использованием отдельного тестового набора данных для снижения влияния переобучения.
2.Потребление памяти и количество операций (миллионы операций умножения-накопления, MMac).
3. Правило обучения: ADAM, размер батча: 200 (единый для всех наборов данных), количество эпох: 150.
4. Начальный learning rate = 0,01, уменьшается в 10 раз на 50% и 75% от общего числа эпох.
5. Обучение выполняется с использованием GPU-сред Google Colab или Kaggle.

Paper results

TABLE 5. Calculation for DCNN configuration at the root for each dataset.

Dataset	Size (KB)	Val Acc. (%)	ΔEG	No. Layer
CIFAR10	87.040	57.0	NA	
SVHN	87.040	86.5	NA	1
EMNIST	105.472	75.0	NA	
CIFAR10	108.544	62.0	0.233	
SVHN	108.544	89.3	0.130	2
EMNIST	164.864	84.5	0.160	
CIFAR10	178.186	78.0	0.230	
SVHN	178.186	96.9	0.109	3
EMNIST	366.592	84.9	0.002	
CIFAR10	230.400	89.5	0.220	
SVHN	230.400	97.0	0.002	4
EMNIST	574.464	85.2	0.001	
CIFAR10	349.184	90.0	0.004	
SVHN	339.978	97.2	0.002	5
EMNIST	784.384	85.4	0.001	
CIFAR10	936.960	91.0	0.002	
SVHN	931.840	98.1	0.002	6
EMNIST	1,382.400	85.6	0.000	

TABLE 6. Investigating variations in test error and number of operations across various datasets and techniques.

Dataset	Method	MMac	Size (MB)	Test Error (%)
CIFAR-10	VGG-16 [2]	313	76.6	6.7
	DenseNet-121 [23]	59	99.6	7.0
	MobileNet V2 [24]	101	8.6	6.0
	***HMDCNN-Tree**	**28**	**0.8**	**7.9**
SVHN	DenseNet-121 [23]	59	99.6	1.7
	Wide ResNet-16,4 [26]	1,935	10.7	1.6
	***HMDCNN-Tree**	**30**	**0.5**	**1.8**
EMNIST	EDEN [25]	23	-	11.7
	Supervised SNN [27]	18,700	-	14.4
	***HMDCNN-Tree**	**4**	**0.3**	**7.8**

- HMDCNN is much lighter in size and MMac
- It beats VGG16 on CIFAR-10 and SVHN, but ResNet18 remains strongest.
- On EMNIST, HMDCNN is close to both baselines with a much smaller model size.

Our implementation results

Baseline model comparison

Dataset	Method	MMac	Size	Err	Val	Acc
CIFAR-10	VGG16	314.57	57.20	38.76%	60.70%	61.24%
CIFAR-10	ResNet18	557.89	42.66	16.82%	83.86%	83.18%
CIFAR-10	**HMDCNN**	16.62	1.66	20.90%	79.10%	79.10%
SVHN	VGG16	314.57	57.20	27.52%	72.18%	72.48%
SVHN	ResNet18	557.89	42.66	5.77%	94.02%	94.23%
SVHN	**HMDCNN**	9.28	0.51	12.91%	88.60%	87.10%
EMNIST	VGG16	314.59	57.27	11.38%	89.21%	88.62%
EMNIST	ResNet18	557.91	42.73	10.57%	89.62%	89.43%
EMNIST	**HMDCNN**	197.74	8.10	11.76%	87.80%	88.20%

- HMDCNN is much lighter in size and MMac
- It beats VGG16 on CIFAR-10 and SVHN, but ResNet18 remains strongest.
- On EMNIST, HMDCNN is close to both baselines with a much smaller model size.

Dataset	Root conv	Tree depth
CIFAR-10	**2**	**3**
EMNIST	**1**	**4**
SVHN	**1**	**2**

Conclusions and future directions

This work proposes **two** modifications of the GEA algorithm, both of which improve its performance compared to the baseline GEA under certain conditions:

- **Variant 1** demonstrates improved performance compared to the original GEA only when a fixed time constraint is known in advance. Under alternative stopping criteria, the results are inconsistent and require deeper theoretical and empirical analysis
- **Variant 2** shows instability in high-dimensional problem settings, limiting its practical applicability

Future work will focus on:

- Evaluating the first approach on real-world combinatorial optimization problems (e.g., GQAP and related models)
- Developing and refining a robust stopping criterion that ensures stable and consistent performance across different scenarios
- Conducting further research to better understand the behavior of both modifications under varying conditions

Thank you!

If you have any question, we’re ready to answer them